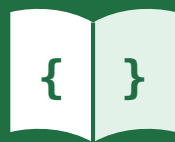

Module 03 — REST APIs with Web MVC

Lesson Notes

Spring Boot Edu



2026-09-23

Contents

1. Learning Goals	2
2. Concepts	2
2.1 REST, HTTP Methods and Status Codes	2
2.2 The Journey of a Request	2
2.3 DTO versus Domain Object	3
3. Step-by-Step Examples	3
3.1 A CRUD Controller	3
3.2 Validation	4
3.3 Errors: ProblemDetail (RFC 9457)	5
3.4 Paging and Sorting	6
3.5 API Versioning (Spring Framework 7)	7
3.6 Content Negotiation	8
3.7 Customising JSON with Jackson 3	9
3.8 OpenAPI and Swagger UI	10
3.9 Web Requests on Virtual Threads	11
4. Common Mistakes and Best Practices	11
5. Summary	12
6. Further Reading	12

1. Learning Goals

By the end of this module you will be able to:

- Write a CRUD REST API with the right HTTP methods and status codes
- Validate requests with Bean Validation
- Return errors in the RFC 9457 ProblemDetail format
- Offer paging and sorting
- Use Spring Framework 7's built-in **API versioning**
- Serve the same resource in different formats through content negotiation
- Customise the JSON output with Jackson 3
- Generate an OpenAPI document and Swagger UI with springdoc
- Handle web requests on virtual threads and test the API with MockMvcTester

Prerequisites: Modules 01, 02 · **Estimated time:** 5 hours

2. Concepts

2.1 REST, HTTP Methods and Status Codes

In a REST API everything is a **resource** (/api/books/1). The HTTP method says what to do, the status code says how it went:

Operation	Method and path	Successful response
List	GET /api/books	200 OK
Get	GET /api/books/{id}	200 OK, or 404 Not Found
Create	POST /api/books	201 Created + Location header
Update	PUT /api/books/{id}	200 OK
Delete	DELETE /api/books/{id}	204 No Content
Invalid request	any	400 Bad Request
Business rule conflict	e.g. out of stock	409 Conflict

2.2 The Journey of a Request

1. DispatcherServlet receives the request.
2. HandlerMapping finds the right controller method by path, method and **version**.
3. Parameters are bound (@PathVariable, @RequestParam, @RequestBody) and validated.
4. The controller returns an object. An HttpMessageConverter (Jackson for JSON) turns it into the format the Accept header asks for.

5. If an exception is thrown, a `@RestControllerAdvice` turns it into an error response.

2.3 DTO versus Domain Object

The API contract (the **DTO**, e.g. `BookResponse`) is kept separate from the internal model (`Book`). Changing the internal model does not break clients. When a change would break clients, a new API version is opened (section 3.5).

3. Step-by-Step Examples

Start the application. Tomcat listens on port 8080:

```
./mvnw -pl modules/03-web-mvc/lesson -am spring-boot:run
```

Send the requests from `modules/03-web-mvc/requests.http` in your IDE, or use the curl commands below. Swagger UI: <http://localhost:8080/swagger-ui.html>

3.1 A CRUD Controller

Goal: list, get, create, update and delete a resource.

The controller is thin: it handles HTTP, delegates the work to `BookService` and returns a DTO:

```
@GetMapping(path =("/{id}", version = "1")
@Operation(summary = "Bir kitabı getir (v1) / Get a book (v1)")
public BookResponse get(@PathVariable @Positive long id) {
    return BookResponse.from(books.find(id));           // BookNotFoundException → 404
    ↪ ProblemDetail
}
```

Creating returns 201 Created and reports the new resource's address in the Location header:

```
@PostMapping
@Operation(summary = "Kitap ekle / Add a book")
public ResponseEntity<BookResponse> create(@Valid @RequestBody BookRequest request) {
    Book created = books.create(request);
    URI location = ServletUriComponentsBuilder.fromCurrentRequest()
        .path("/{id}").buildAndExpand(created.id()).toUri();
    return ResponseEntity.created(location).body(BookResponse.from(created)); // 201 +
    ↪ Location header
}
```

Run it:

```
curl -s localhost:8080/api/books/1
curl -s -i -X POST localhost:8080/api/books -H 'Content-Type: application/json' \
  -d '{"isbn": "978-0-13-235088-4", "title": "Clean Code", "authors": ["Robert C.
    ↪ Martin"], "price": 75.50, "publishedOn": "2008-08-01"}'
```

Expected output:

```
{"id":1,"isbn":"9780134685991","title":"Effective Java","author":"Joshua
  ↪ Bloch","price":89.90,"publishedOn":"2018-01-06"}

HTTP/1.1 201
Location: http://localhost:8080/api/books/6
{"id":6,"isbn":"9780132350884","title":"Clean Code","author":"Robert C.
  ↪ Martin","price":75.50,"publishedOn":"2008-08-01"}
```

Its test: `book/BookControllerTest` is a web slice test. `@WebMvcTest` loads only the MVC infrastructure and the given controller, and `MockMvcTester` makes the tests read like `AssertJ`:

```
@Test
void getsABook() {
    assertThat(mvc.get().uri("/api/books/1"))
        .hasStatusOk()
        .bodyJson().isLenientlyEqualTo("""
    ↪ {"id": 1, "isbn": "9780134685991", "title": "Effective Java",
        "author": "Joshua Bloch",
        "price": 89.90, "publishedOn": "2018-01-06"}""");
}
```

i Note

In Spring Boot 4 the test slices live in technology-specific test starters. `@WebMvcTest` needs `spring-boot-starter-webmvc-test`.

3.2 Validation

Goal: never let invalid data reach the controller.

The rules live on the DTO. `@ISBN` is Hibernate Validator's constraint that also checks the ISBN checksum:

```
public record BookRequest(
    @NotBlank @ISBN @Nullable String isbn,                // checks the
    ↪ ISBN checksum too
    @NotBlank @Size(max = 200) @Nullable String title,
    @NotEmpty @Nullable List<@NotBlank String> authors,    // constraints on
    ↪ list elements
    @NotNull @Positive @Digits(integer = 6, fraction = 2) @Nullable BigDecimal price,
    @NotNull @PastOrPresent @Nullable LocalDate publishedOn) {
}
```

In the controller, `@Valid @RequestBody` validates the body. **Parameter constraints** such as `@PathVariable @Positive long id` or `@RequestParam @Max(50) int size` work through Spring MVC's built-in method validation, without any extra setup.

Run it:

```
curl -s -X POST localhost:8080/api/books -H 'Content-Type: application/json' \
  -d '{"isbn": "123", "title": "", "authors": [], "price": -5}'
```

Expected output:

```
{"detail": "One or more fields are
↪ invalid", "instance": "/api/books", "status": 400, "title": "Invalid request",
"errors": [{"field": "authors", "message": "must not be
↪ empty"}, {"field": "isbn", "message": "invalid ISBN"},
{"field": "price", "message": "must be greater than 0"}, {"field": "title", "message": "must
↪ not be blank"},
{"field": "publishedOn", "message": "must not be null"}]}
```

3.3 Errors: ProblemDetail (RFC 9457)

Goal: use a single, standard, machine-readable format for every error.

With `spring.mvc.problemDetails.enabled: true`, Spring MVC's own errors come back as `application/problem+json`. For domain errors we write a `@RestControllerAdvice`:

```
@ExceptionHandler
ProblemDetail handleBookNotFound(BookNotFoundException exception, HttpServletRequest
↪ request) {
    ProblemDetail problem = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND,
↪ exception.getMessage());
    problem.setType(URI.create("https://springbootedu.com/problems/book-not-found"));
    problem.setTitle("Book not found");
    problem.setInstance(URI.create(request.getRequestURI()));
}
```

```
        problem.setProperty("bookId", exception.bookId());           // extra, machine-readable
    ↪    field
        return problem;
    }
```

To tell the client which fields are wrong, override the matching method of `ResponseEntityExceptionHandler`:

```
@Override
protected @Nullable ResponseEntity<Object> handleMethodArgumentNotValid(
    MethodArgumentNotValidException exception, HttpHeaders headers, HttpStatusCode
    ↪    status, WebRequest request) {
    ProblemDetail problem = ProblemDetail.forStatusAndDetail(status, "One or more fields
    ↪    are invalid");
    problem.setTitle("Invalid request");
    List<Map<String, String>> errors =
        ↪    exception.getBindingResult().getFieldErrors().stream()
            .map(error -> Map.of("field", error.getField(), "message",
                ↪    String.valueOf(error.getDefaultMessage())))
            .toList();
    problem.setProperty("errors", errors);                             // tell the client WHICH
    ↪    fields are wrong
    return handleExceptionInternal(exception, problem, headers, status, request);
}
```

Expected output (`curl -s -i localhost:8080/api/books/99`):

```
HTTP/1.1 404
Content-Type: application/problem+json
{"detail":"No book with id 99","instance":"/api/books/99","status":404,"title":"Book not
    ↪    found",
 "type":"https://springbootedu.com/problems/book-not-found","bookId":99}
```

★ Tip

type is a stable URI that identifies the kind of error. Clients should decide based on type, not on the message text.

3.4 Paging and Sorting

Goal: return large lists piece by piece, and protect the server from oversized requests.

```

@GetMapping
@Operation(summary = "Kitapları sayfa sayfa listele / List books page by page")
public PageResponse<BookResponse> list(
    @RequestParam(defaultValue = "0") @Min(0) int page,
    @RequestParam(defaultValue = "20") @Min(1) @Max(50) int size,           // protect
    ↪ the server
    @RequestParam(defaultValue = "title") @Pattern(regexp =
    ↪ "title|price|publishedOn") String sort,
    @RequestParam(defaultValue = "asc") @Pattern(regexp = "asc|desc") String
    ↪ direction) {
    PageResponse<Book> result = books.findPage(page, size,
    ↪ BookService.SortField.from(sort), direction.equals("desc"));
    return new PageResponse<>(result.content().stream().map(BookResponse::from).toList(),
        result.page(), result.size(), result.totalElements(), result.totalPages());
}

```

Run it: `curl -s 'localhost:8080/api/books?page=1&size=2&sort=price&direction=desc'`

Expected output:

```

{"content":[{"id":1,...,"title":"Effective Java",...,"price":89.90,...},
            {"id":5,...,"title":"Refactoring",...,"price":85.00,...}],
"page":1,"size":2,"totalElements":5,"totalPages":3}

```

i Note

Here we write paging by hand. In Module 06 we will use Spring Data's Pageable and Page types.

3.5 API Versioning (Spring Framework 7)

Goal: publish a new, incompatible version of an API without breaking existing clients.

Spring Framework 7 supports versioning out of the box. In Boot, the settings live in `application.yaml`:

```

mvc:
  problemdetails:
    enabled: true           # Lesson 3.3 – Spring MVC's own errors as
    ↪ application/problem+json
  apiversion:              # Lesson 3.5 – Spring Framework 7 API versioning
    use:
      header: API-Version  # the client chooses the version with this request header
      supported: 1, 2
      default: 1           # requests without the header get version 1

```

Two versions of the same URL are told apart with the version attribute. v1 is the get method (section 3.1), v2 looks like this:

```
@GetMapping(path =("/{id}", version = "2")           // selected with the header
↳ "API-Version: 2"
@Operation(summary = "Bir kitabı getir (v2) / Get a book (v2)")
public BookResponseV2 getV2(@PathVariable @Positive long id) {
    return BookResponseV2.from(books.find(id));
}
```

In v2 the authors are a list and the price is an object with a currency. That change would break v1 clients:

```
public record BookResponseV2(long id, Isbn isbn, String title, List<String> authors,
↳ Price price, LocalDate publishedOn) {

    public record Price(BigDecimal amount, String currency) {
    }
}
```

Run it:

```
curl -s -H 'API-Version: 2' localhost:8080/api/books/2
curl -s -H 'API-Version: 7' localhost:8080/api/books/1
```

Expected output:

```
{ "id": 2, "isbn": "9780321336781", "title": "Java Puzzlers", "authors": ["Joshua Bloch", "Neal
↳ Gafter"],
  "price": { "amount": 55.00, "currency": "TRY", "publishedOn": "2005-07-04" }

{ "detail": "Invalid API version:
↳ '7.0.0'.", "instance": "/api/books/1", "status": 400, "title": "Bad Request" }
```

★ Tip

Instead of a header, the version can also travel in the path (use `.path-segment`), a query parameter (use `.query-parameter`) or the media type (use `.media-type-parameter`). Pick one approach and use it across the whole API.

3.6 Content Negotiation

Goal: serve the same resource in different formats, depending on the client's Accept header.

A second GET `/api/books` method with `produces = "text/csv"` is chosen for clients that send `Accept: text/csv`:

```
@GetMapping(produces = "text/csv")                // same URL, chosen by the Accept
↪ header
public String exportCsv() {
    return books.findAll().stream()
        .map(book -> book.id() + "," + book.isbn().value() + "," + book.title() + ","
            ↪ + book.price())
        .collect(Collectors.joining("\n", "id,isbn,title,price\n", "\n"));
}
```

Run it: `curl -s -H 'Accept: text/csv' localhost:8080/api/books`

Expected output:

```
id,isbn,title,price
1,9780134685991,Effective Java,89.90
2,9780321336781,Java Puzzlers,55.00
...
```

3.7 Customising JSON with Jackson 3

Goal: change how a value object looks in JSON.

Spring Boot 4 uses **Jackson 3**. The package name changed from `com.fasterxml.jackson` to `tools.jackson`, and dates are written in ISO format (`"2018-01-06"`) by default. `Isbn` is a record and would normally be written as `{"value": "..."}.` With a `@JacksonComponent` it is written and read as a plain string:

```
@JacksonComponent
public class IsbnJacksonComponent {

    public static class Serializer extends ValueSerializer<Isbn> {

        @Override
        public void serialize(Isbn isbn, JsonGenerator generator, SerializationContext
            ↪ context) {
            generator.writeString(isbn.value());
        }
    }

    public static class Deserializer extends ValueDeserializer<Isbn> {

        @Override
        public Isbn deserialize(JsonParser parser, DeserializationContext context) {
```

```
        return new ISBN(parser.getString()); // Jackson 3: getString() replaces
        ↪ getText()
    }
}
}
```

Expected output: "isbn":"9780134685991" in the response of section 3.1.

! Warning

Careful when copying Jackson 2 examples: JsonSerializer → ValueSerializer, SerializerProvider → SerializationContext, getText() → getString(), and JacksonException is now unchecked.

3.8 OpenAPI and Swagger UI

Goal: generate a machine-readable contract and interactive documentation of the API automatically.

The springdoc-openapi-starter-webmvc-ui dependency scans the controllers. The @Tag and @Operation descriptions are added to the document, and the general information comes from an OpenAPI bean:

```
@Configuration(proxyBeanMethods = false)
public class OpenApiConfiguration {

    @Bean
    OpenAPI bookstoreOpenApi() {
        return new OpenAPI().info(new Info()
            .title("Kitapçı API / Bookstore API")
            .version("v1, v2 (header: API-Version)")
            .description("Spring Boot Edu – module 03"));
    }
}
```

Open: <http://localhost:8080/swagger-ui.html> (UI) and <http://localhost:8080/v3/api-docs> (JSON contract)

Its test: docs/OpenApiTest

i Note

springdoc is not in the Spring Boot BOM. Its version is kept in one place, build-parent/pom.xml (3.1.1, built for Boot 4.1).

3.9 Web Requests on Virtual Threads

Goal: see what `spring.threads.virtual.enabled=true` does in Tomcat.

```
@RestController
public class ThreadInfoController {

    public record ThreadInfo(String name, boolean virtual) {
    }

    @GetMapping("/api/runtime/thread")
    public ThreadInfo current() {
        Thread thread = Thread.currentThread();
        return new ThreadInfo(thread.toString(), thread.isVirtual());
    }
}
```

Run it: `curl -s localhost:8080/api/runtime/thread`

Expected output:

```
{"name": "VirtualThread[#63,tomcat-handler-15]/runnable@ForkJoinPool-1-worker-1", "virtual": true}
```

Its test: `runtime/VirtualThreadsIT` **really** starts the application on a random port and calls it with Spring Framework 7's `RestTestClient`. `MockMvc` cannot test this, because it does not use a real server.

4. Common Mistakes and Best Practices

!! Caution

Do not return domain objects (e.g. JPA entities) directly as JSON. Internal fields leak out, and every change to the internal model breaks clients. Always use a DTO.

- **Do:** return 201 Created with Location when creating, and 204 No Content when deleting.
- **Don't:** return errors as `{"success": false}` inside a 200 OK body. Use the right status code and `ProblemDetail`.
- **Do:** put an upper limit on the page size (`@Max(50)`).
- **Don't:** write business logic in the controller. Keep controllers thin and do the work in the service layer.
- **Do:** open a new API version for incompatible changes and keep supporting the old one for a while.

- **Don't:** leave tests that change stateful (in-memory) data depending on each other. Refresh the test context with `@DirtiesContext` or reset the data in every test.

5. Summary

- In a REST API, methods and status codes carry meaning. Use 201 + Location, 204, 404 and 409 correctly.
- Bean Validation works on DTOs and on method parameters.
- `ProblemDetail` is a standard contract for every error.
- Spring Framework 7 offers API versioning out of the box, through the version attribute and a few settings.
- Content negotiation serves different formats from the same URL. In Jackson 3, customisation is done with `@JacksonComponent`.
- `springdoc` generates the contract automatically. `MockMvcTester` and `RestTestClient` test the API.

Next step: [Exercises](#)

6. Further Reading

- [Spring Framework — Request Mapping](#)
- [Spring Framework — API Versioning](#)
- [Spring Framework — Error Responses \(ProblemDetail\)](#)
- [Spring Framework — Validation](#)
- [Spring Framework — Content Negotiation](#)
- [Spring Framework — MockMvcTester](#)
- [Spring Boot — Servlet Web Applications · JSON](#)
- [RFC 9457 — Problem Details for HTTP APIs · springdoc-openapi](#)